

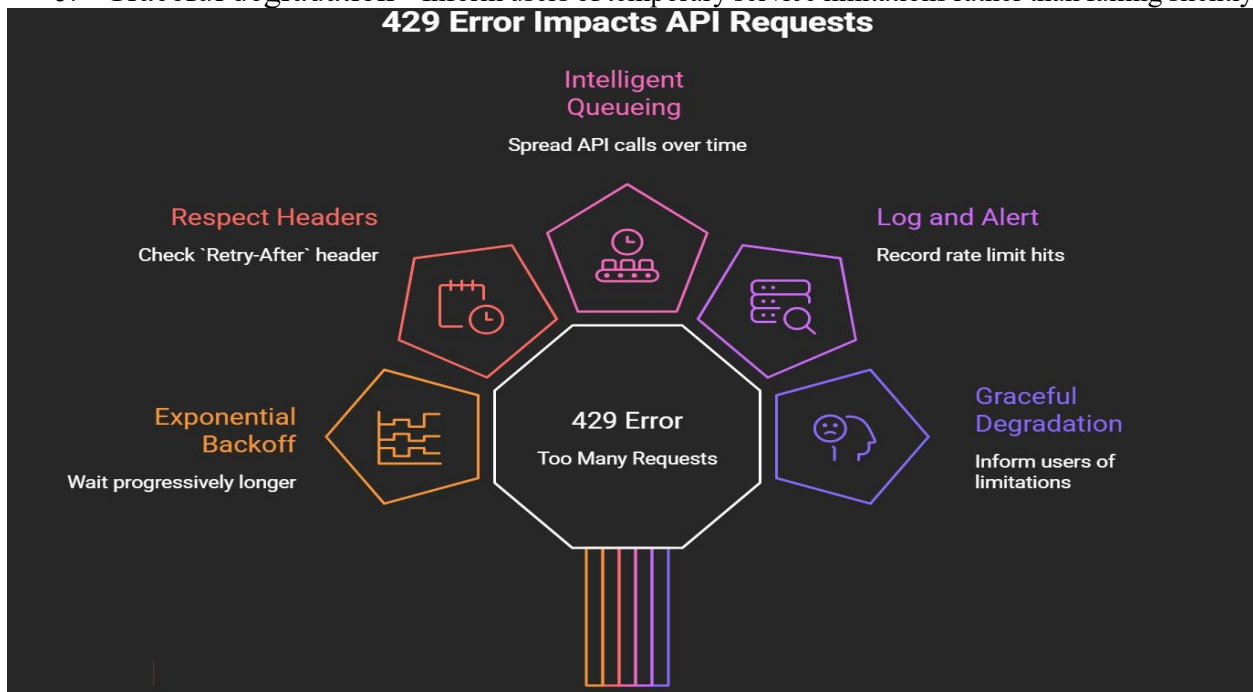
Mini Assessment

Name	CHAGAMREDDY HARINADH REDDY
Registration No	99230040275
Year	IV
Department	CSE(DS)

Question 1: If an API returns 429, what should your program do?

Answer: A 429 status code indicates "Too Many Requests" (rate limiting). Your program should:

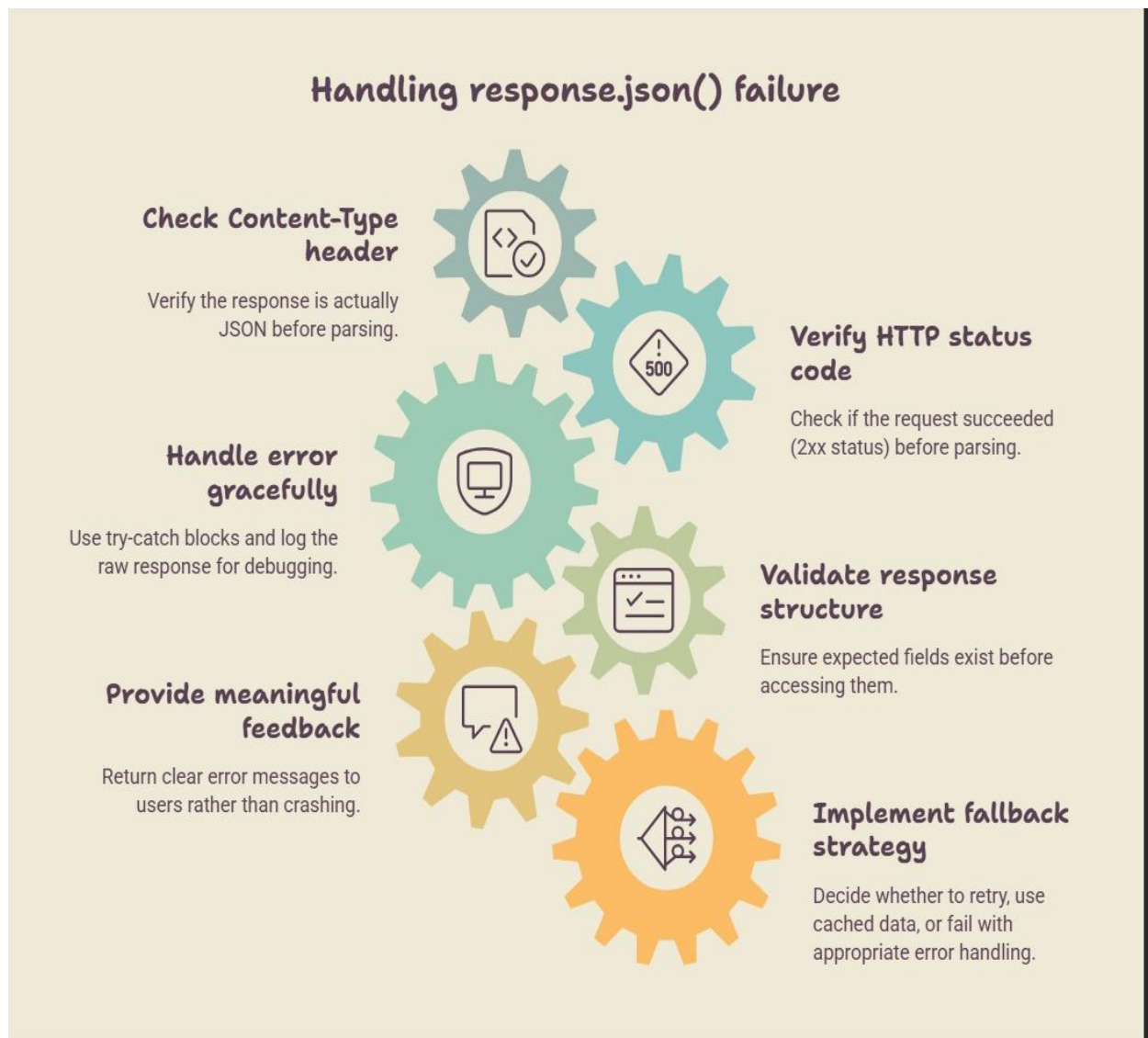
1. Implement exponential backoff - Wait progressively longer before retrying (e.g., 1s, 2s, 4s, 8s)
2. Respect rate limit headers - Check the Retry After response header to determine exact wait time
3. Queue requests intelligently - Spread API calls over time rather than sending them all at once
4. Log and alert - Record rate limit hits for monitoring and adjust request frequency
5. Graceful degradation - Inform users of temporary service limitations rather than failing silently



Question 2: If response.json() fails, what should your program do?

This indicates the response body is not valid JSON. Your program should:

1. Check the Content-Type header - Verify the response is actually JSON before parsing
2. Verify HTTP status code - Check if the request succeeded (2xx status) before parsing
3. Handle the error gracefully - Use try-catch blocks and log the raw response for debugging
4. Validate response structure - Ensure expected fields exist before accessing them
5. Provide meaningful feedback - Return clear error messages to users rather than crashing
6. Fallback strategy - Decide whether to retry, use cached data, or fail with appropriate error handling



Question 3: If a JSON key is missing, why might `.get()` be safer than `[]`?

Aspect	[] (Square Brackets)	.get() Method
Missing key	Throws KeyError	Returns None (or default value)

Aspect	[] (Square Brackets)	.get() Method
Reliability	Crashes program	Continues execution
Defensive coding	Requires explicit checking	Handles absence naturally
Readability	Less intent-clear	Explicit safety intent

Best Practice Example:

Safer approach

```
value = data.get('optional_field', 'default_value')
```

Instead of risky

```
value = data['optional_field'] # Crashes if missing
```

Using `.get()` enables defensive programming, allowing your application to handle incomplete or varying API responses without crashing.

